AI vs Human Readers for Glaucoma Detection

A Bayesian Diagnostic Meta-Analysis in R — From Spreadsheet to Publication

Ting-Wei Wang, MD, PhD
Taipei Veterans General Hospital

2026-04-26

Why are we here?

By the end of this lecture, you should be able to read — not just admire — a Bayesian DTA meta-analysis pipeline.

What you'll take home

1. **Why** clinicians should care about meta-analysis methodology, not just headline numbers
2. **How** to go from a messy Excel workbook to a publication-grade figure
3. **What** the Bayesian hierarchical bivariate model actually does (without the algebra panic)
4. **Where** the common pitfalls hide — and how this notebook handles them

> **Note**
>
> You don't need to memorise the JAGS syntax. You need to recognise the *shape* of the workflow.

The clinical question

> *Across the published literature, **how does AI compare to human readers** for detecting glaucoma — and how confident should we be in the answer?*

This is **not** a single-study question. It's a question about *the body of evidence*.

That's what meta-analysis is for.

Frequentist vs Bayesian bivariate DTA

Both modern frameworks handle between-study variance **and** Se–Sp correlation. The differences are practical:

Feature	Frequentist (Reitsma,)	Bayesian (this notebook)
Between-study variance	✓ Random effects (REML)	✓ Explicit τ posterior
Se–Sp correlation	✓ Bivariate normal	✓ Bivariate normal
Small/sparse data	REML can fail to converge	Priors stabilise estimation
Pr(AI > Reader)	Indirect, requires bootstrap	Native from posterior draws
Prediction intervals	Extra steps	Direct from MCMC draws
>2 nodes / network meta-analysis	Harder, fewer tools	Scales naturally in JAGS

Bottom line: for this two-node design either framework works. We chose Bayesian for the **direct probability statements** (“Pr(AI Se >

The two-node design

```
flowchart LR
    A[Eligible studies] --> B[Extract 2x2 tables<br/>per arm]
    B --> C{Arm type}
    C -->|AI-only| D[AI node]
    C -->|Human-only| E[Reader node]
    D --> F[Bayesian bivariate<br/>DTA model]
    E --> F
    F --> G[Pooled Se, Sp,<br/>contrast, HSROC]
```

Arm-based, not study-based — every eligible 2×2 contributes, even when the study reports only one arm type.

The pipeline at a glance

Six stages, one notebook

1. **Import** → Excel workbook with 7 sheets
2. **Clean** → harmonise modality, region, validation, vendor
3. **Describe** → PRISMA, QUADAS-3, descriptives
4. **Model** → Bayesian bivariate DTA in JAGS
5. **Diagnose** → $\hat{R}$, n.eff, trace plots, heterogeneity
6. **Communicate** → forest, HSROC, GRADE-DTA, sensitivity

> **Tip**

Stage 1: Setup

The R toolkit

```
 1 library(tidyverse)   # data wrangling: dplyr, tidyr, ggplot2
 2 library(readxl)      # read Excel sheets
 3 library(janitor)     # clean column names
 4 library(R2jags)      # the Bayesian engine
 5 library(coda)        # MCMC diagnostics
 6 library(lattice)     # MCMC visual checks
 7 library(gt)          # publication-grade tables
 8 library(patchwork)   # combine plots
 9 library(ggplot2)     # the grammar of graphics
10 library(scales)      # axis formatting
11 library(forcats)     # factor handling
12 library(stringr)     # string ops for harmonisation
13 library(digest)      # cache hashing
```

Three families: wrangling (tidyverse), modelling (R2jags), communication (gt + ggplot2 + patchwork).

The Lancet-style theme

```r
LANCET_COLS <- c(
  "#2166AC", "#D6604D", "#35978F",
  "#6BAED6", "#9E7BB5", "#E6AB5B",
  "#B2182B", "#969696"
)

GLAUCOMA_TWO_NODE <- setNames(
  c(LANCET_COLS[1], LANCET_COLS[2]),
  c("Reader only", "AI only")
)
```

> **Note**
>
> **Lesson:** define the palette *once*, reuse everywhere. Reviewers notice colour consistency.

Stage 2: Data import & cleaning

The workbook structure

Sheet	Role
`03_Studies`	Study-level metadata (year, region, design)
`05_Primary_Nodes`	Pre-specified primary 2×2 tables
`06_Secondary_AllValid`	All eligible arms (what we analyse)
`07_QUADAS3`	Risk-of-bias judgements
`11_PRISMA_Working`	Screening flow counters
`12_Analysis_Ready`	Final analysis table
`14_Match_Audit`	Cohort-pairing audit trail

Why so many sheets? Reproducibility. Every decision — inclusion, harmonisation, pairing — leaves a trail.

Importing in one block

```r
DATA_FILE <- list.files(pattern = "glaucoma_meta_analysis_WORKING_.*\\.xlsx$",
                        full.names = FALSE)[1]

raw_studies   <- read_excel(DATA_FILE, sheet = "03_Studies")   |> clean_names()
raw_secondary <- read_excel(DATA_FILE, sheet = "06_Secondary_AllValid") |> clean_names()
raw_quadas3   <- read_excel(DATA_FILE, sheet = "07_QUADAS3")   |> clean_names()
raw_prisma    <- read_excel(DATA_FILE, sheet = "11_PRISMA_Working") |> clean_names()

cat("Sheets:", paste(excel_sheets(DATA_FILE), collapse = ", "), "\n")
```

`clean_names()` from **janitor** turns `Study Design` into `study_design` — small thing, huge time saver.

Harmonisation: the unglamorous heart of every meta-analysis

```r
harmonise_modality <- function(x) {
  x_clean <- str_to_lower(str_squish(as.character(x)))
  case_when(
    str_detect(x_clean, "fundus\\+oct|oct\\+fundus") ~ "Fundus+OCT",
    str_detect(x_clean, "\\boct\\b")                 ~ "OCT",
    str_detect(x_clean, "fundus")                    ~ "Fundus",
    TRUE                                             ~ "Mixed/Other"
  ) |> factor(levels = c("Fundus", "OCT", "Fundus+OCT", "Mixed/Other"))
}
```

> ⚠ **Warning**
>
> **Pitfall:** “OCT”, “oct”, “OCT imaging”, and “spectral-domain OCT” all mean the same thing. If you don’t harmonise upstream, your subgroup analysis silently fragments.

Deriving the accuracy metrics

```r
derive_accuracy <- function(df) {
  df |>
    mutate(
      n_total      = tp + fp + tn + fn,
      prevalence   = (tp + fn) / n_total,
      sensitivity  = tp / (tp + fn),
      specificity  = tn / (tn + fp),
      ppv          = if_else((tp + fp) > 0, tp / (tp + fp), NA_real_),
      npv          = if_else((tn + fn) > 0, tn / (tn + fn), NA_real_),
      youden       = sensitivity + specificity - 1
    )
}
```

Every row now carries: prevalence, Se, Sp, PPV, NPV, Youden. **One source of truth.**

What “arm-based” means in practice

```r
df_arm_all <- raw_secondary |>
  filter(node_type %in% c("AI-only", "Human-only")) |>     # only the two we model
  mutate(across(c(tp, fn, tn, fp), as.numeric)) |>
  filter(!if_any(c(tp, fn, tn, fp), is.na)) |>             # complete 2x2 only
  left_join(study_lookup, by = "study_id") |>
  mutate(
    cohort_id    = coalesce(as.character(analysis_block_id),
                            as.character(study_id)),       # cohort = unit of analysis
    node_compare = harmonise_node(node_type)
  ) |>
  derive_accuracy()
```

A *cohort* may contribute one node or two. The model handles both — that’s what hierarchical means.

Stage 3: Describing what we have

PRISMA flow — built from a key-value sheet

```r
prisma_lookup <- setNames(raw_prisma$value, raw_prisma$metric)
prisma_records <- as.numeric(prisma_lookup[["records_in_screening_sheet"]])

# Then: ggplot rectangles + arrows, no special package needed
# (see notebook for full plotting code)
```

Tip

Lesson: PRISMA flow is just four boxes and three arrows. You don't need a dedicated package — `geom_rect` + `annotate("segment", ...)` does it cleanly.

PRISMA flow — our study

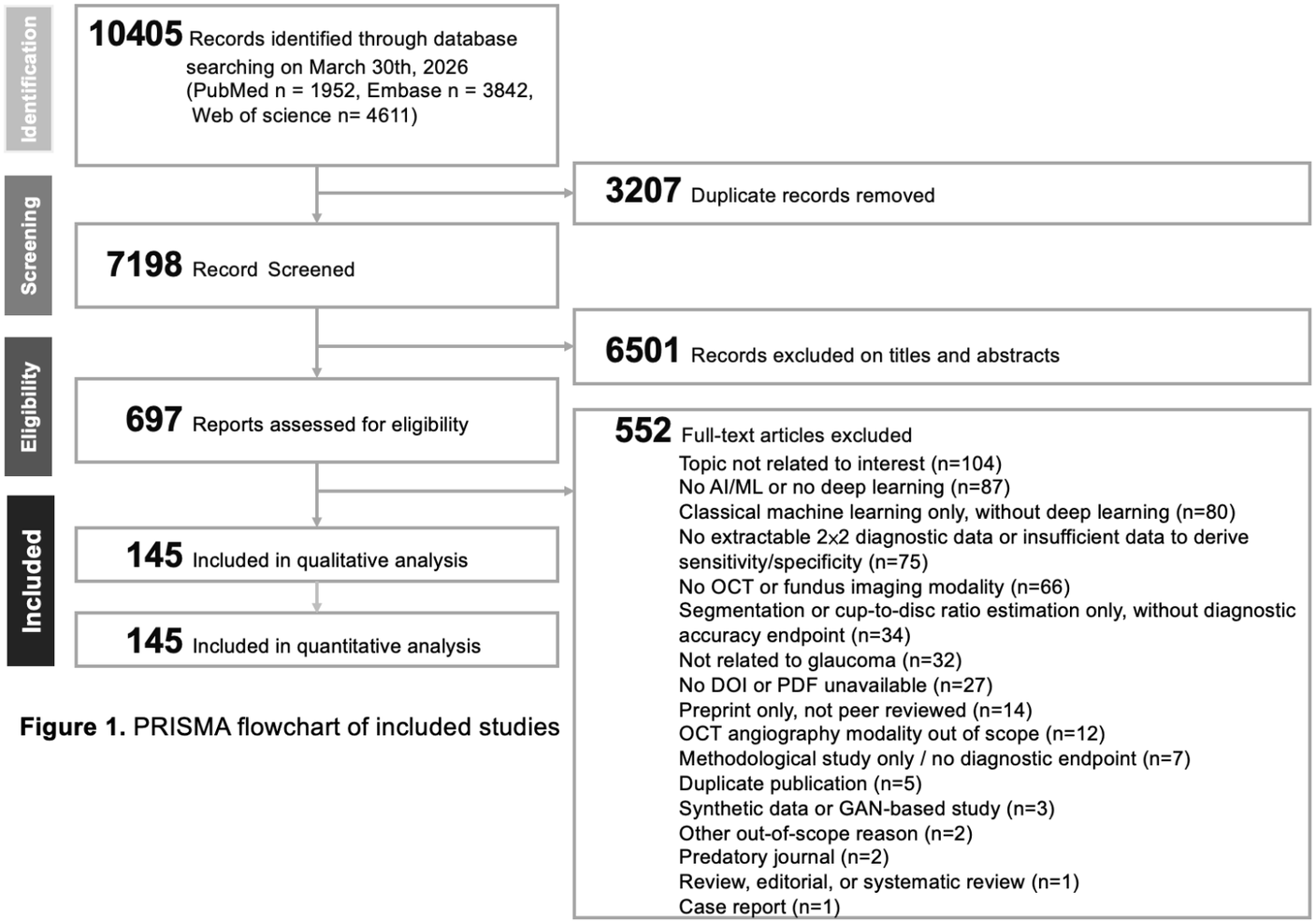


Figure 1. PRISMA flowchart of included studies

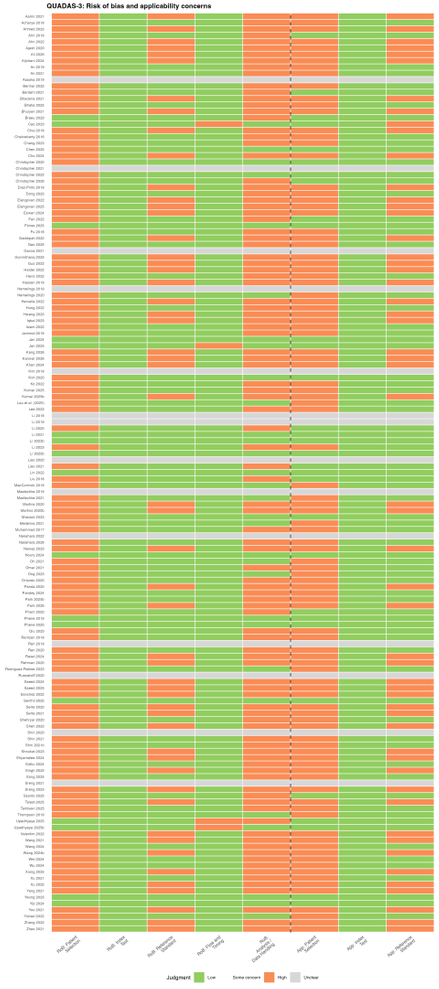
Figure 1. Identification → screening → eligibility → inclusion. 697 records → 151 included.

QUADAS-3 traffic light

```r
q3_long <- raw_quadas3 |>
  pivot_longer(-study_label, names_to = "domain", values_to = "judgment") |>
  mutate(Judgment_std = case_when(
    str_detect(str_to_lower(judgment), "high")    ~ "High",
    str_detect(str_to_lower(judgment), "concern") ~ "Some concern",
    str_detect(str_to_lower(judgment), "low")     ~ "Low",
    TRUE                                          ~ "Unclear"))

rob_cols <- c("Low" = "#91CF60", "Some concern" = "#FEE08B",
              "High" = "#FC8D59", "Unclear" = "#D9D9D9")

ggplot(q3_long, aes(domain, study_label, fill = Judgment_std)) +
  geom_tile(colour = "white") + scale_fill_manual(values = rob_cols)
```

Standard. Familiar. Reviewers expect it.

QUADAS-3 — our study



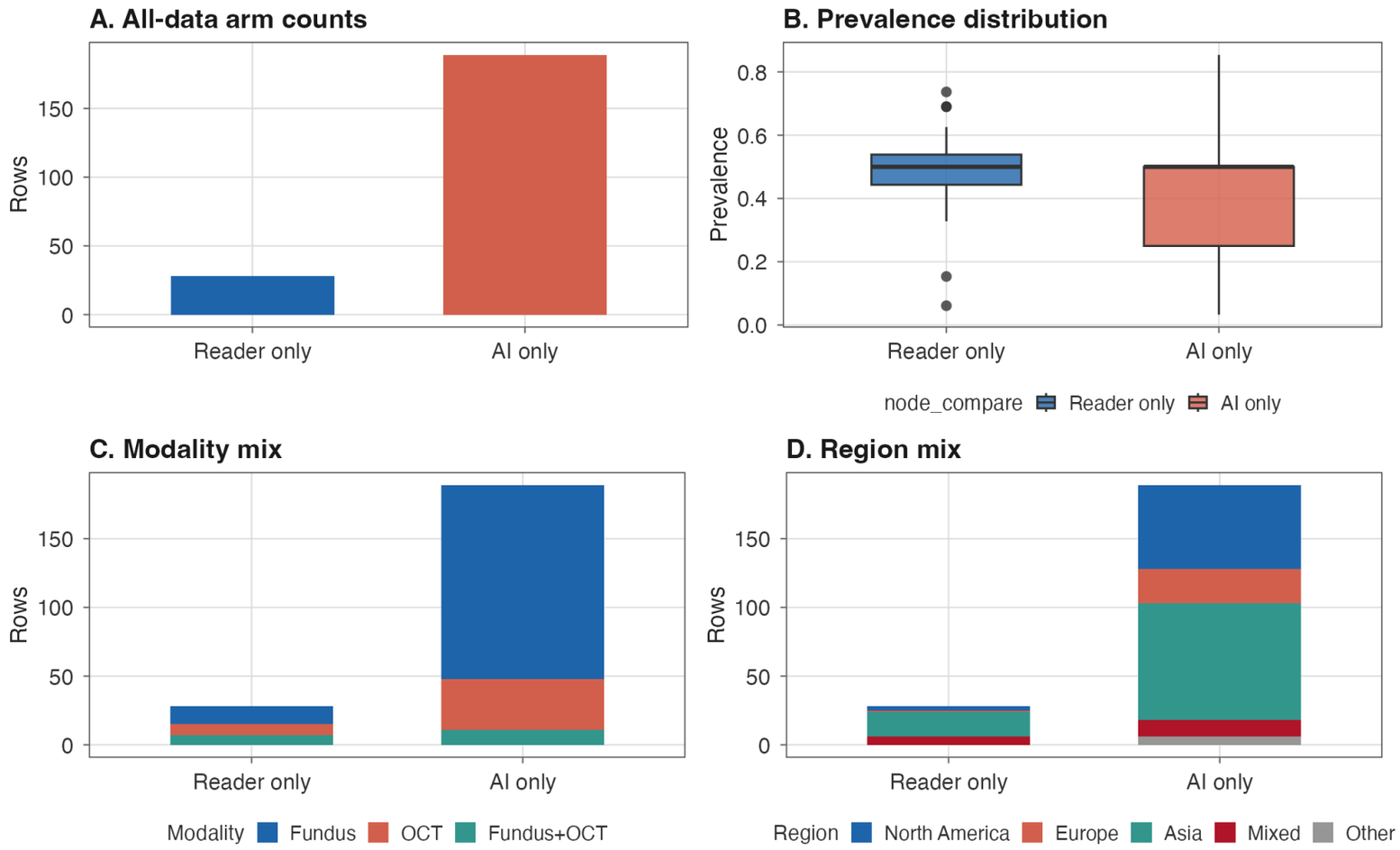
eFigure 9. Risk-of-bias judgements per study × QUADAS-3 domain.

Descriptive panel

- Node counts (how many AI-only vs Reader-only rows)
- Prevalence distribution (boxplot per node)
- Modality mix (Fundus / OCT / Both)
- Region mix (geographic balance)

> **Note**
>
> This panel goes in the **supplement**, not the main paper. But you'll be glad you made it during peer review.

Descriptive panel — our study



eFigure 1. Node counts, prevalence, modality, and region distributions.

Stage 3b: Transitivity

Transitivity — can we compare AI and Reader arms at all?

The assumption: every covariate that shifts Se or Sp is distributed *similarly* across the AI-only and Reader-only cohort pools.

Why it matters here: our design is **arm-based**, not study-based. AI cohorts and Reader cohorts are mostly *different* studies. The AI–Reader contrast therefore inherits every covariate that differs between those two pools — not just the intervention.

> **Important**
>
> **Two nodes ≠ no transitivity problem.** With only direct evidence the *consistency* test (direct vs indirect agreement) is unavailable. That makes the *transitivity* check more important, not less.

The DTA effect modifiers we screen for:

- **Target condition** (glaucoma, GON, referable, POAG, suspect)
- **Reference standard** (specialist, OCT-RNFL, perimetry, composite)
- **Modality** (fundus, OCT, both)
- **Validation type** (internal vs external)
- **Spectrum / prevalence** (screening vs referral)

Transitivity in *our* data — the asymmetry to flag

Effect modifier	AI-only (n=189)	Human-only (n=28)
Modality — fundus / OCT / both	75% / 20% / 6%	46% / 29% / 25%
Validation — internal / external	66% / 34%	**21% / 79%**
Target — glaucoma / GON / other	76% / 14% / 10%	50% / 25% / 25%
Cohorts	150	16

Read the validation row. AI evidence is dominated by internal validation; Reader evidence by external validation. A naïve AI–Reader contrast partly compares *internal-AI* to *external-reader* — and internal validation flatters AI.

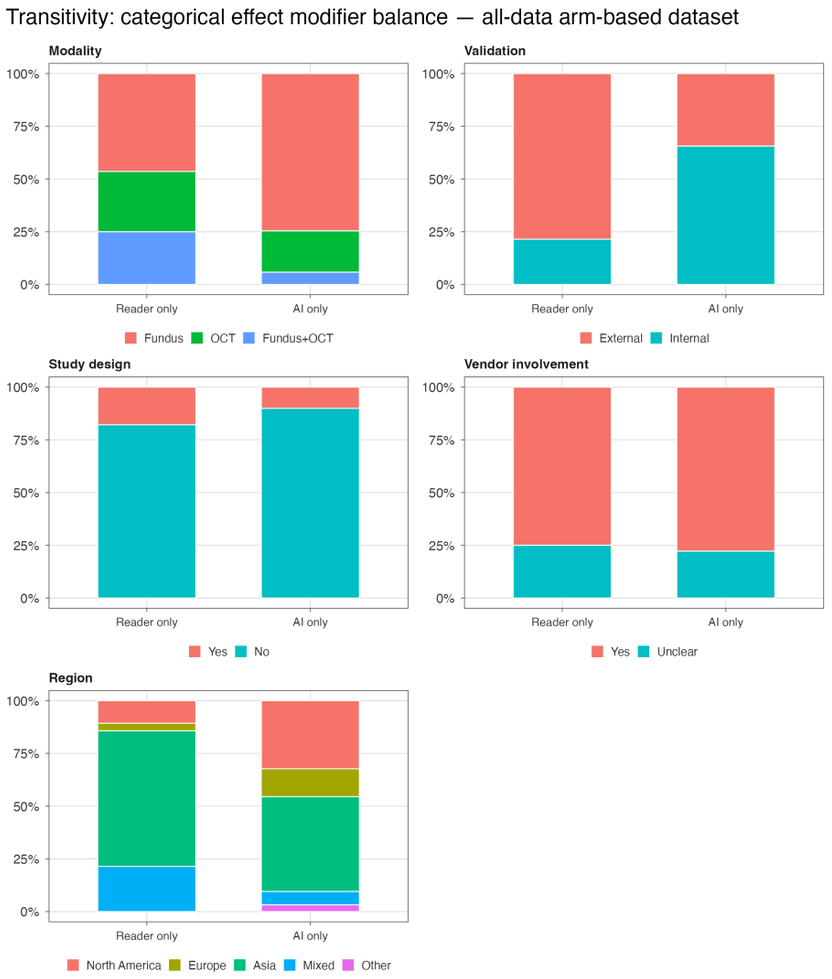
> ⚠ **Warning**
>
> **Reviewer-facing fix:** declare each asymmetry; report **external-only** as a co-primary; add modality + validation + target as meta-regression covariates.

How we check it — and what we do if it fails

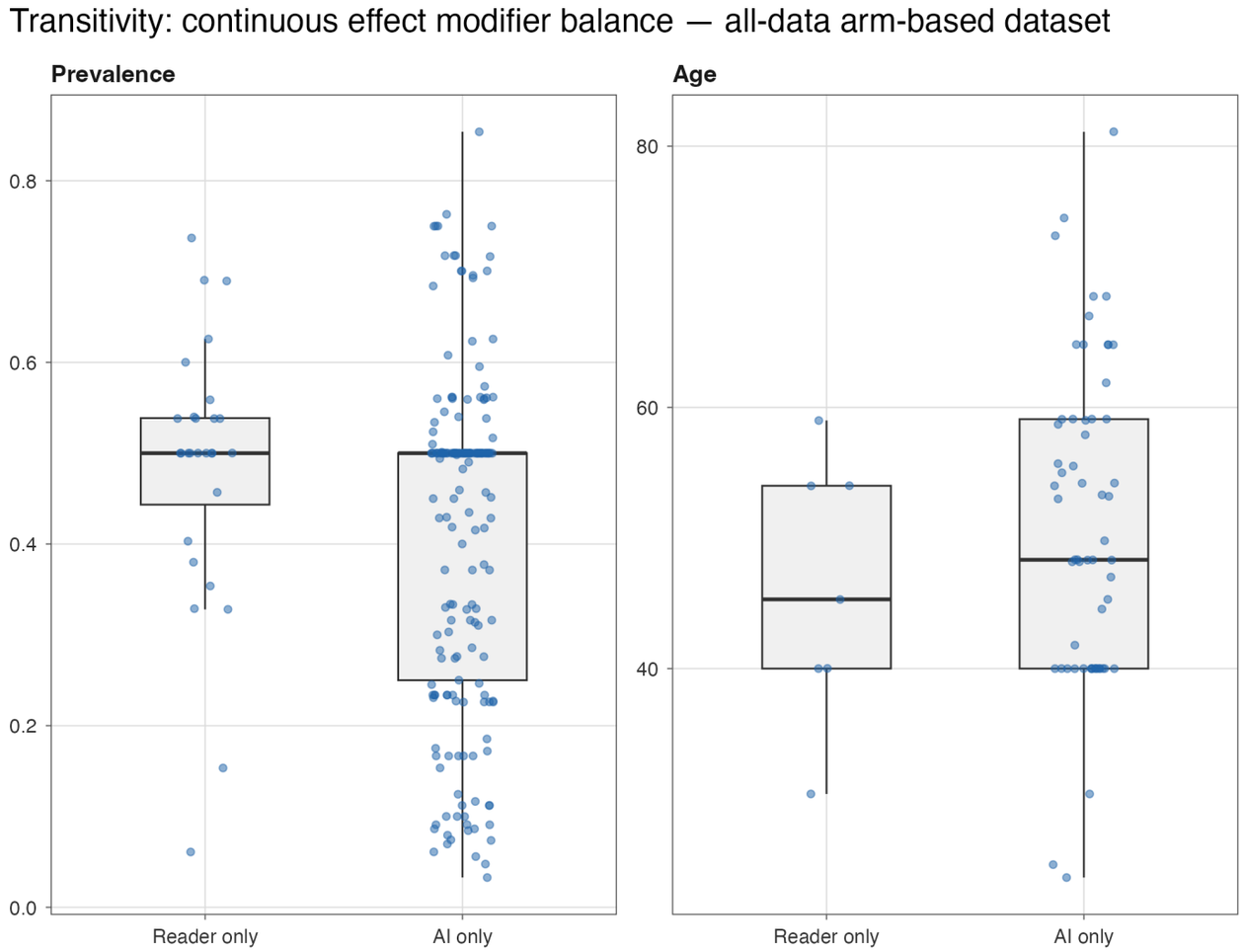
```r
# 1. Descriptive: tabulate effect modifiers per node, side-by-side
transitivity_table <- df_arm_all |>
  count(node_compare, modality_std, validation_std, target_std) |>
  pivot_wider(names_from = node_compare, values_from = n, values_fill = 0)

# 2. Empirical: meta-regression on the suspected modifiers
fit_se <- lm(logit_safe(sensitivity) ~ node_compare + modality_std +
             validation_std + target_std + scale(prevalence),
             data = mv_df, weights = tp)

# 3. Restriction: refit the Bayesian model on a transitivity-balanced subset
df_external_only <- df_arm_all |> filter(validation_std == "External")
```

- **Plausible** → main analysis stands; side-by-side table in supplement.
- **Borderline** → include modifier as covariate in JAGS, report adjusted contrast.
- **Violated** → restrict the network (external-only) and report *that* as the primary contrast.

Transitivity — categorical effect modifiers



Transitivity — continuous effect modifiers



Stage 4: The Bayesian model

The model — in plain words

For each row i from cohort s, test type k:

$$\text{TP}_i \sim \text{Binomial}(\pi_{i,1}, \text{positive}_i)$$

$$\text{TN}_i \sim \text{Binomial}(\pi_{i,2}, \text{negative}_i)$$

$$\text{logit}(\pi_{i,1}) = \mu_k^{Se} + u_s^{Se} + v_{s,k}^{Se}$$

$$\text{logit}(\pi_{i,2}) = \mu_k^{Sp} + u_s^{Sp} + v_{s,k}^{Sp}$$

Note

Three layers of variation: - μ_k: the test-type mean (AI vs Reader) - u_s: between-cohort heterogeneity (correlated Se/Sp) - $v_{s,k}$: within-cohort × test-type residual

The JAGS code — observation layer

```
for(i in 1:nObs) {
    tp[i] ~ dbin(pi[i,1], pos[i])
    tn[i] ~ dbin(pi[i,2], neg[i])
    logit(pi[i,1]) <- threshold.sens[test[i],1]
                    + study.re[s[i],1]
                    + test.re.sens[s[i],test[i]]
    logit(pi[i,2]) <- threshold.spec[test[i],1]
                    + study.re[s[i],2]
                    + test.re.spec[s[i],test[i]]
}
```

Read it left to right: **observed counts → binomial → logit-scale linear predictor**.

The JAGS code — random effects

```
 1  for(k in 1:ns) {
 2      study.re[k,1:2] ~ dmnorm(zero2[], Tau.study[,])
 3      for(l in 1:ntest) {
 4        test.re.sens[k,l] ~ dnorm(0, tautestsens)
 5        test.re.spec[k,l] ~ dnorm(0, tautestspec)
 6      }
 7  }
 8  Sigma.study[1,1] <- pow(SDstudysens,2)
 9  Sigma.study[2,2] <- pow(SDstudyspec,2)
10  Sigma.study[1,2] <- rho.study * SDstudysens * SDstudyspec
11  Sigma.study[2,1] <- Sigma.study[1,2]
12  rho.study ~ dunif(-1, 1)
```

Important

The `dmnorm` line is what makes this **bivariate** — it lets sensitivity and specificity correlate within a cohort, which is the whole point.

The priors — weakly informative

```
threshold.sens[k,1] ~ dnorm(0, 0.05)   # vague on logit scale
threshold.spec[k,1] ~ dnorm(0, 0.05)
SDstudysens ~ dunif(0, 2)
SDstudyspec ~ dunif(0, 2)

# precision = 0.05 means SD ≈ 4.5 on logit scale
# → 95% prior on Se covers (0.01, 0.99)
# → genuinely uninformative within plausible range
```

Question to anticipate from reviewers: *Did the priors drive the result?*

The answer should be **no, and here's the prior-predictive check** (in the supplement).

Fitting the model

```
data_list <- list(
  ns = n_distinct(model_df$study_num),
  ntest = 2,
  nObs = nrow(model_df),
  s = model_df$study_num,  test = model_df$test_num,
  tp = model_df$tp,        tn = model_df$tn,
  pos = model_df$tp + model_df$fn,
  neg = model_df$fp + model_df$tn
)

mod <- jags(data = data_list, inits = make_inits_list(...),
            parameters.to.save = parameters,
            n.chains = 4, n.iter = 20000, n.burnin = 10000,
            model.file = textConnection(model_string))
```

Tip

Cache your fits. The notebook hashes the data + model string + iterations and reuses cached `.rds` — saves hours during revisions.

Stage 5: Diagnostics

Did the chains converge?

The two checks every Bayesian paper needs:

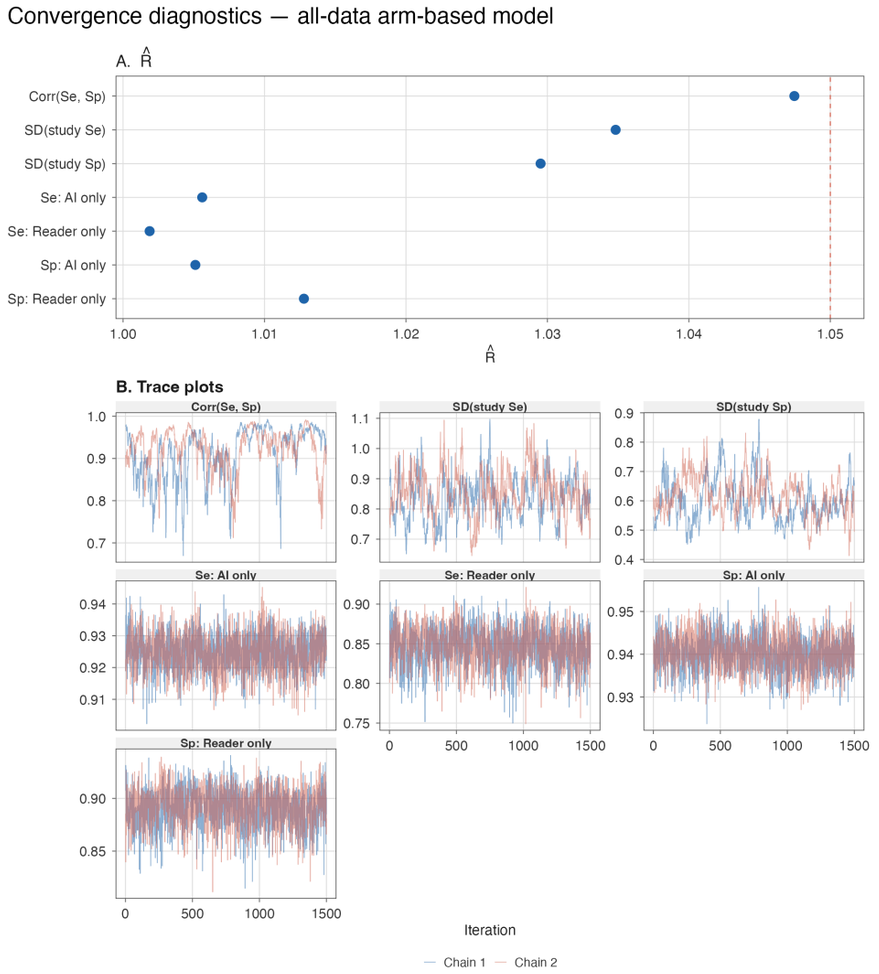
- $\hat{R} \leq 1.05$ for all monitored parameters
- Effective sample size $n_{\text{eff}} \geq 400$

Visual confirmation:

- Trace plots: chains should look like *fuzzy caterpillars*
- No drift, no stuck regions, full overlap

> ⚠ **Warning**
>
> **Red flag:** $\hat{R} > 1.1$ on any parameter → don't interpret the posterior. Increase iterations or reparameterise.

Convergence — our trace plots



Heterogeneity — the τ table

Parameter	Interpretation
τ (study Se)	Between-cohort SD on logit-Se
τ (study Sp)	Between-cohort SD on logit-Sp
τ (test Se)	Test-type residual on Se
τ (test Sp)	Test-type residual on Sp
ρ (Se–Sp correlation)	Within-cohort Se/Sp linkage

Clinically: large τ → pooled estimate is a *summary*, not a *prediction* for your next patient. Always report the **prediction interval** alongside the credible interval.

Stage 6: Communicating the answer

Pooled performance — the table

```r
extract_performance_summary <- function(model_obj, model_df, node_levels) {
  feat <- model_obj$BUGSoutput$summary
  se_rows <- feat[grep("^sens\\[", rownames(feat)), ]
  sp_rows <- feat[grep("^spec\\[", rownames(feat)), ]

  tibble(
    Node = node_levels,
    Studies = map_int(seq_along(node_levels),
                      ~length(unique(model_df$cohort_id[model_df$test_num == .x]))),
    Sensitivity = sprintf("%.3f [%.3f, %.3f]",
                          se_rows,"mean"], se_rows,"2.5%"], se_rows,"97.5%"]),
    Specificity = sprintf("%.3f [%.3f, %.3f]",
                          sp_rows,"mean"], sp_rows,"2.5%"], sp_rows,"97.5%"])
  )
}
```

AI vs Reader — the contrast

```r
contrast_ai_vs_reader <- function(model_obj, label) {
  sims <- model_obj$BUGSoutput$sims.list
  inv_logit <- function(x) 1 / (1 + exp(-x))
  se_diff <- inv_logit(sims$threshold.sens[, 2, 1]) -
             inv_logit(sims$threshold.sens[, 1, 1])
  sp_diff <- inv_logit(sims$threshold.spec[, 2, 1]) -
             inv_logit(sims$threshold.spec[, 1, 1])
  tibble(
    Se_diff = sprintf("%.3f [%.3f, %.3f]",
                      mean(se_diff), quantile(se_diff, 0.025), quantile(se_diff, 0.975)),
    Pr_Se_superior = mean(se_diff > 0),
    Pr_Sp_superior = mean(sp_diff > 0)
  )
}
```

Pr(AI Se > Reader Se) is the number clinicians actually want — and it falls out of MCMC for free.

Forest plot — pooled estimates with prediction intervals

- **Thick line:** 95% credible interval for the pooled mean
- **Thin line:** 95% **prediction** interval for a *new* cohort
- **Two metrics:** sensitivity (left), specificity (right)
- **Two nodes:** AI only (top), Reader only (bottom)

> **Important**
>
> The thin line is usually the surprise. *“Pooled Se = 0.92 [0.89, 0.94]”* sounds great — until you see the prediction interval is **0.78–0.97**. That’s the honest message.

Forest plot — our study

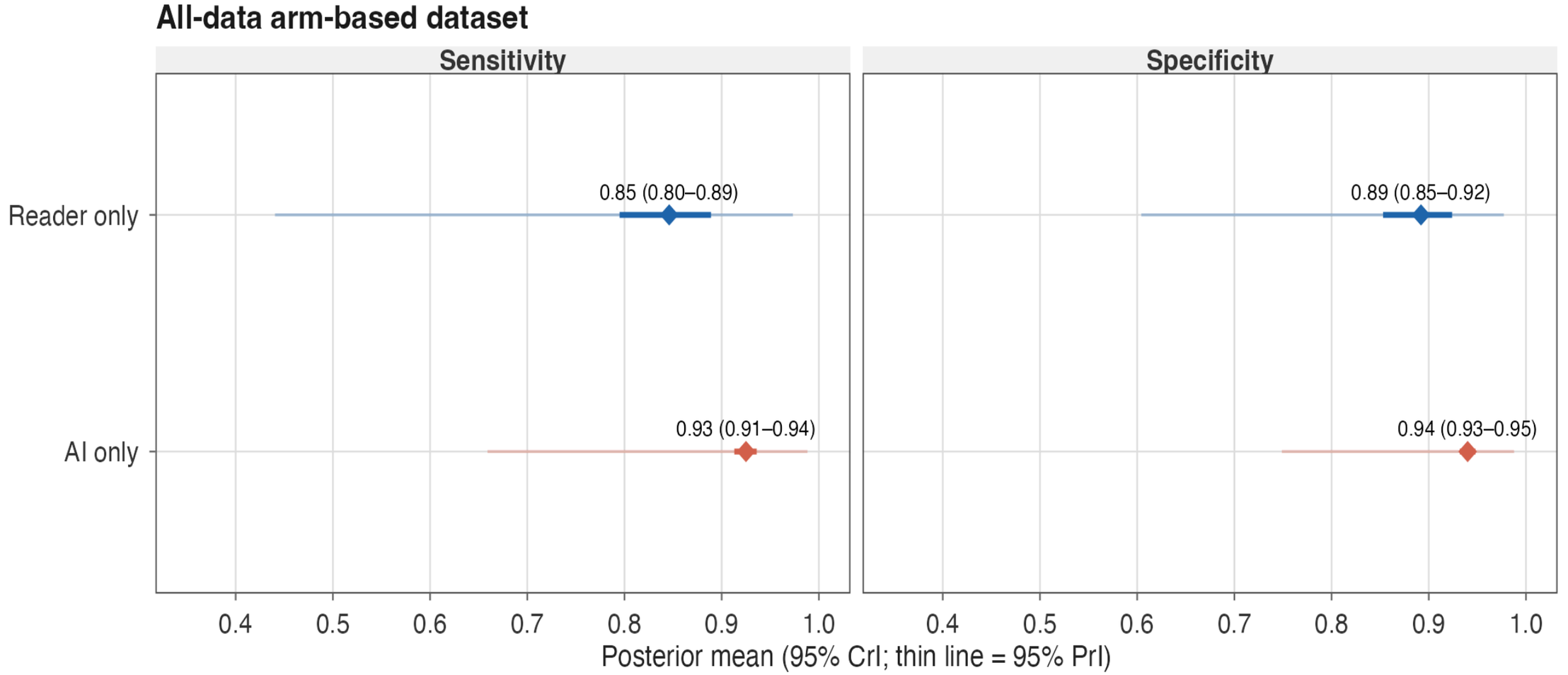


Figure 2. Pooled Se and Sp with 95% credible (thick) and prediction (thin) intervals, AI vs Reader.

HSROC — the bivariate space

- Each **dot** = one cohort's observed (1–Sp, Se)
- **Solid ellipse** = 95% credible region for the pooled mean
- **Dashed ellipse** = 50% credible region (tighter visualisation)
- **Two ellipses** in different colours = AI vs Reader

> **Note**
>
> HSROC is the right plot for DTA meta-analysis. ROC curves from a single threshold model would be misleading here.

HSROC — our study

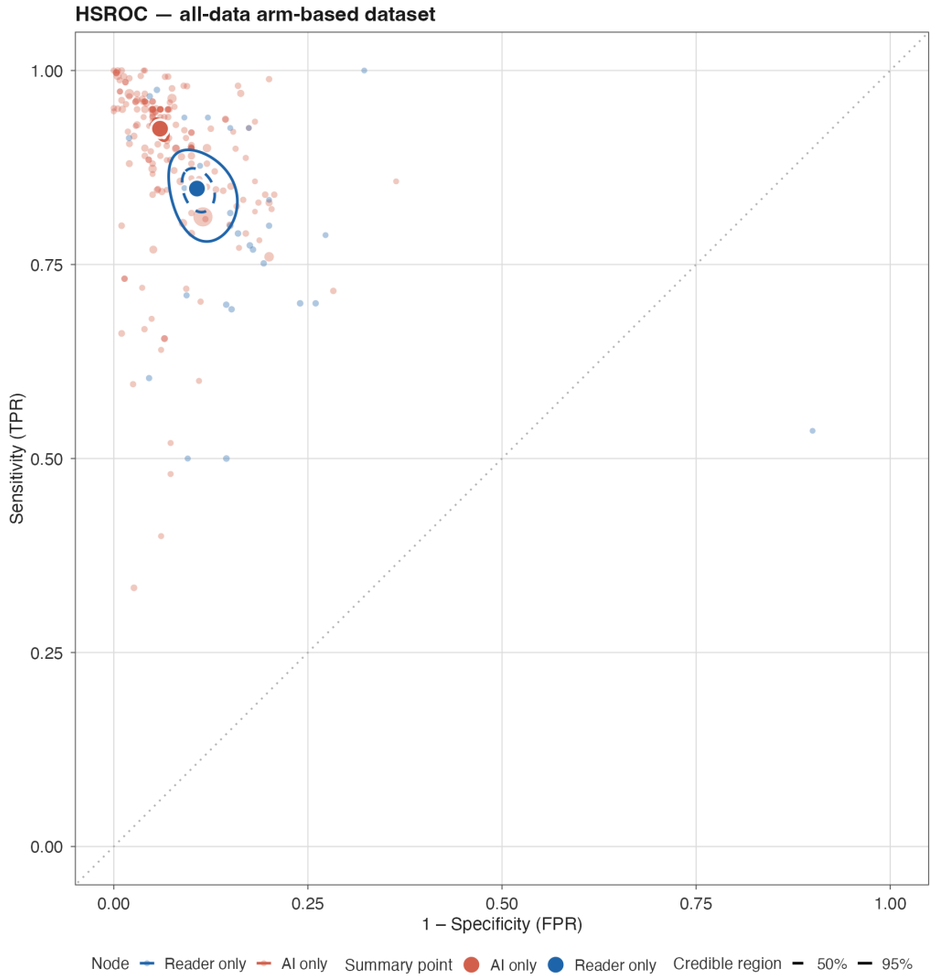


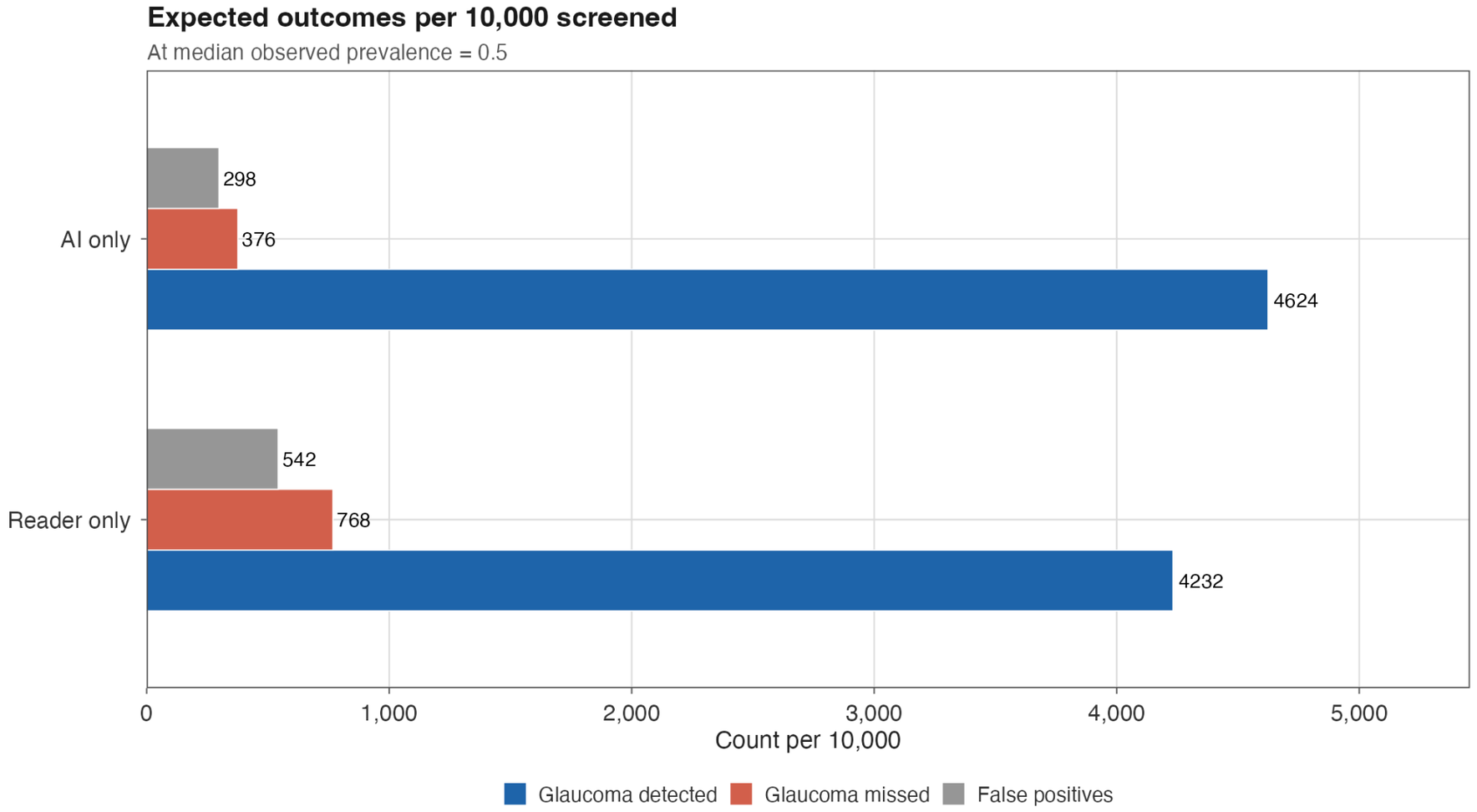
Figure 3. Each dot = one cohort; ellipses = 95% / 50% credible regions per node.

Clinical impact — speak the language

```r
median_prev <- median(df_arm_all_model$prevalence, na.rm = TRUE)
clinical_impact <- arm_all_summary |>
  transmute(
    Node, Se = Se_mean, Sp = Sp_mean,
    TP_10k = round(Se * median_prev * 10000),
    FN_10k = round((1 - Se) * median_prev * 10000),
    FP_10k = round((1 - Sp) * (1 - median_prev) * 10000)
  )
```

Per 10,000 screened: how many glaucoma cases caught, how many missed, how many false alarms. **This is the slide your audience remembers.**

Clinical impact — our numbers



eFigure 2. TP / FN / FP per 10,000 screened at the cohort median prevalence.

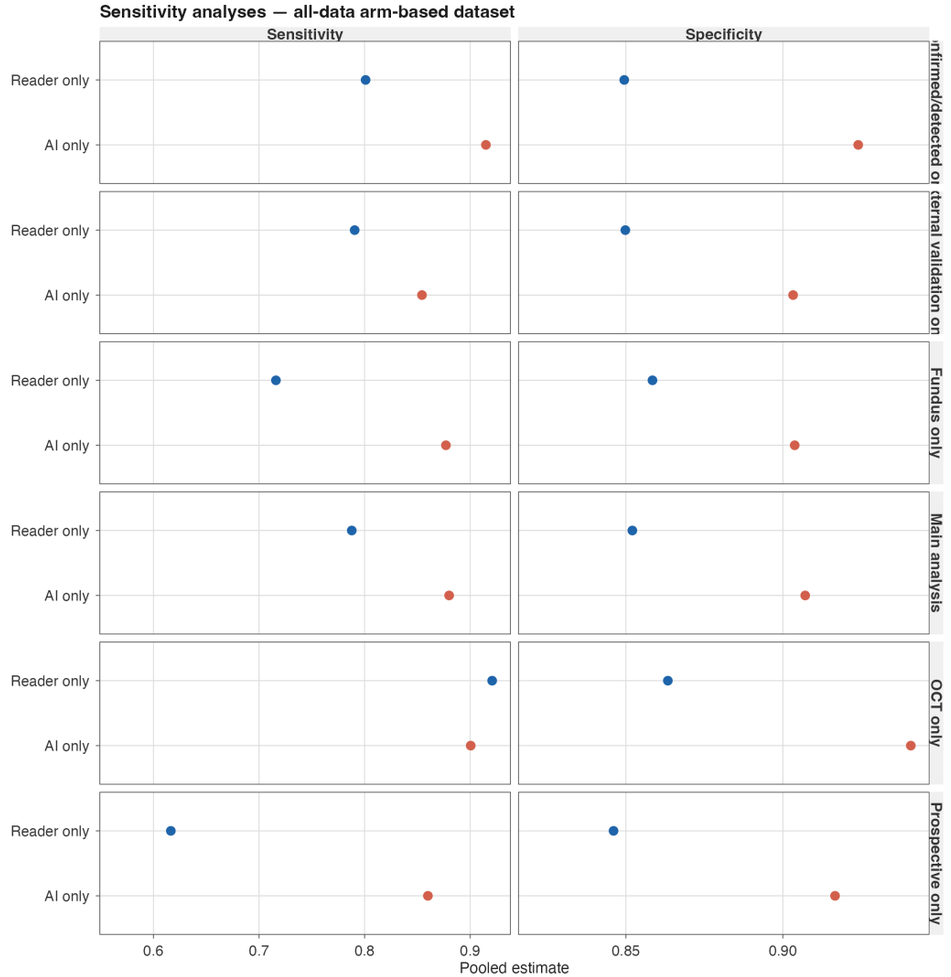
Stage 6b: Robustness

Sensitivity analyses — pre-specified

- **Confirmed/detected only** — drop “referable” target
- **External validation only** — gold-standard generalisability
- **No vendor involvement** — independence check
- **Prospective only** — design-based bias
- **Fundus only / OCT only** — modality stratification

> **Tip**
>
> **Lesson:** decide these *before* unblinding the model. Post-hoc sensitivity analyses are the leading cause of reviewer scepticism.

Sensitivity analyses — our study



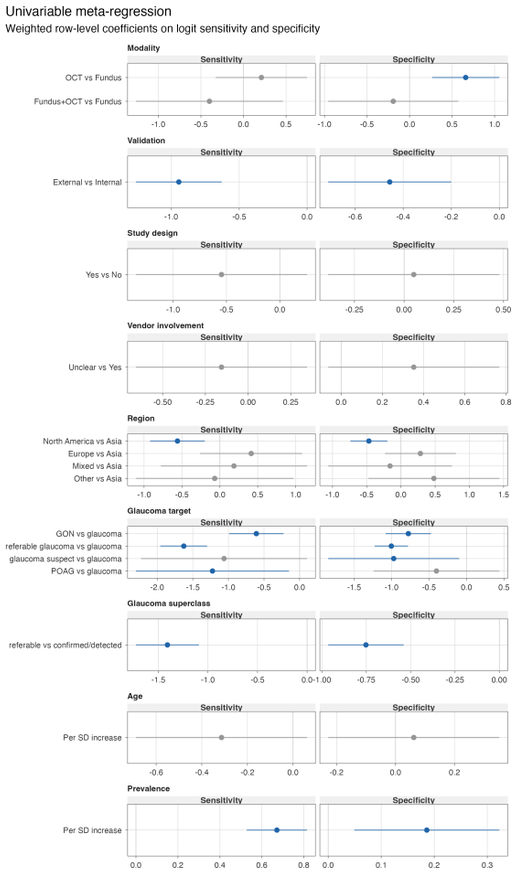
eFigure 6. Pooled Se / Sp under each pre-specified restriction. Stable = robust.

Meta-regression — what drives heterogeneity?

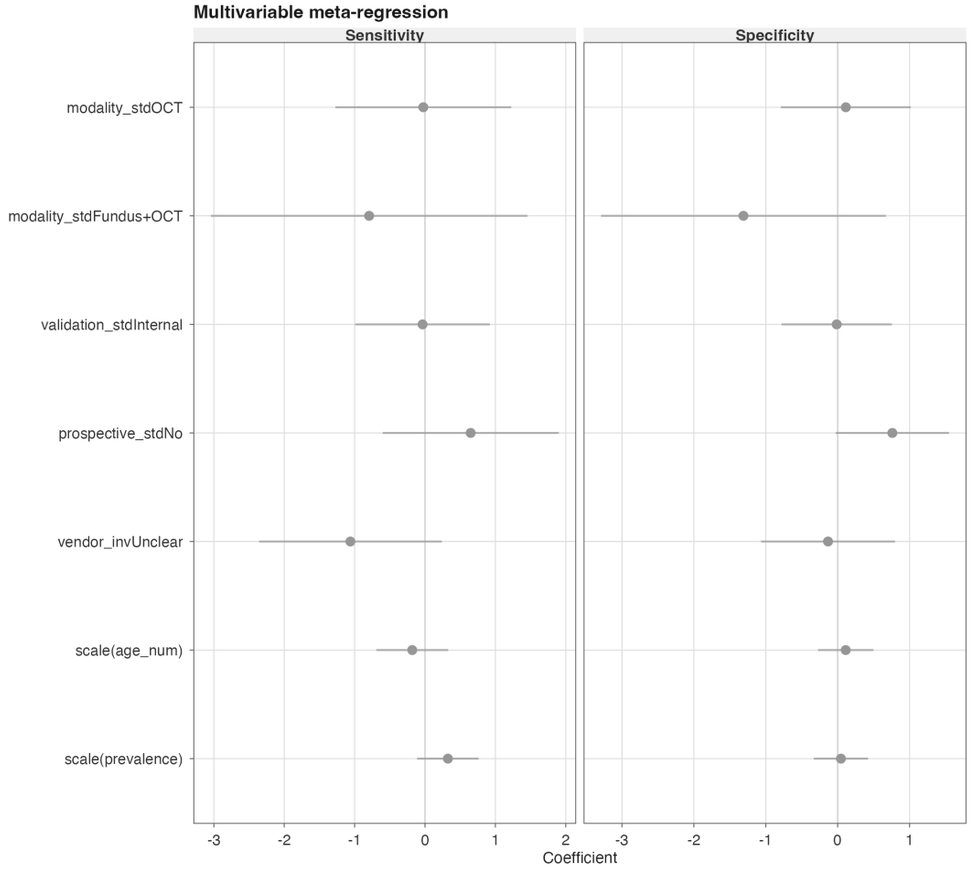
```r
fit_mv_se <- lm(
  logit_safe(sensitivity) ~ modality_std + validation_std +
    prospective_std + vendor_inv + scale(age_num) + scale(prevalence),
  data = mv_df, weights = tp
)
# weights = TP for sensitivity model
# weights = TN for specificity model
# logit_safe handles 0/1 boundaries
```

Weighted regression on the logit scale, weights = the count that “anchors” each metric.

Meta-regression — our coefficients



eFigure 4. Univariable.



eFigure 5. Multivariable.

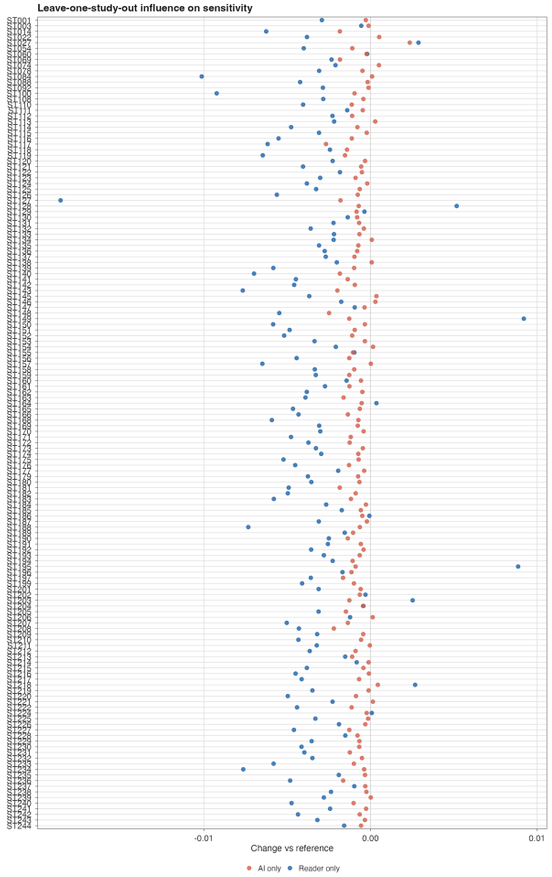
Leave-one-study-out

For each study j, refit the model on all studies *except* j, and plot the change in pooled Se and Sp.

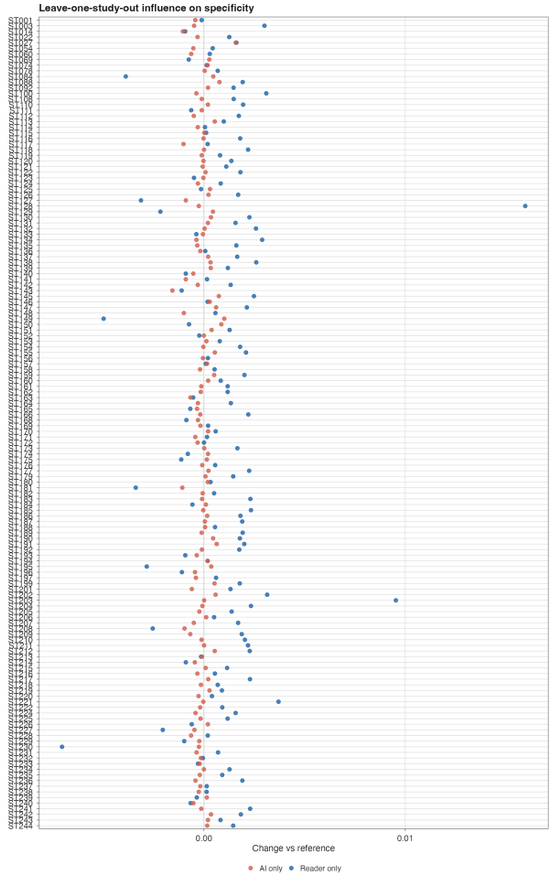
> ⚠ **Warning**
>
> **What you're looking for:** any single study moving the pooled estimate by > 0.02–0.03. That's a study driving the result.

In a healthy meta-analysis, the LOO plot is a **cloud near zero** with no outliers.

LOO — our study



eFigure 13. ΔSe per study.



eFigure 14. ΔSp per study.

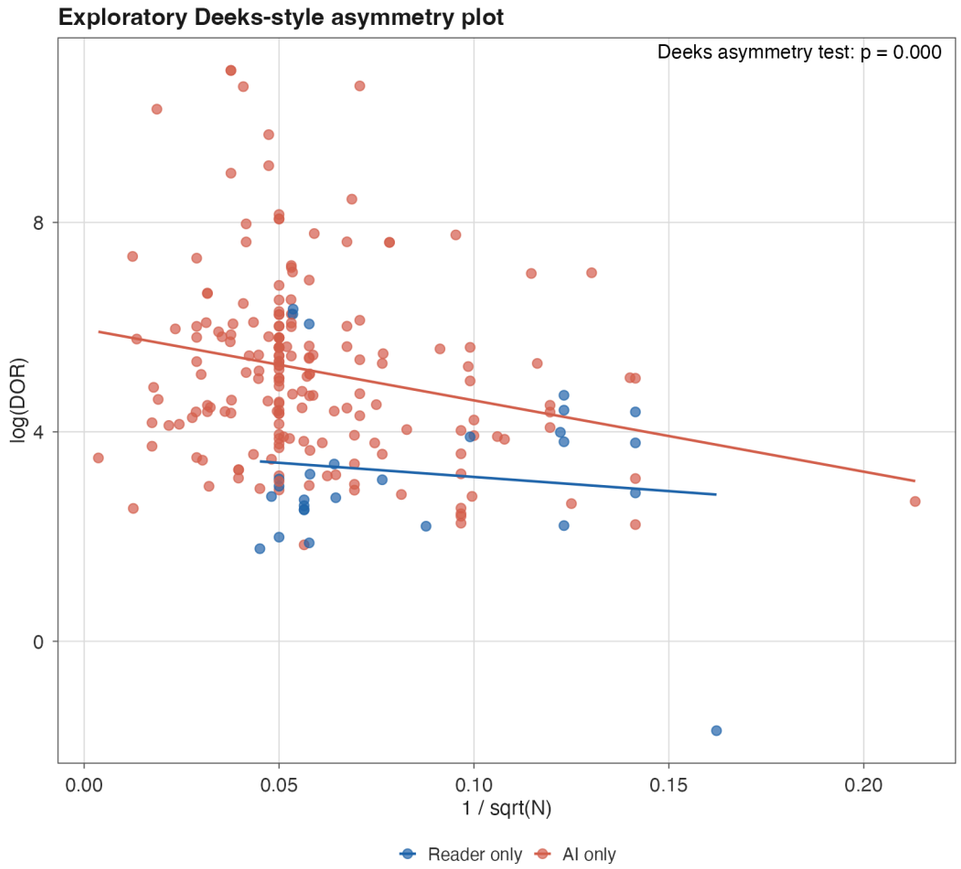
Deeks asymmetry — publication bias

$$\log(\text{DOR}) \sim \frac{1}{\sqrt{N}}$$

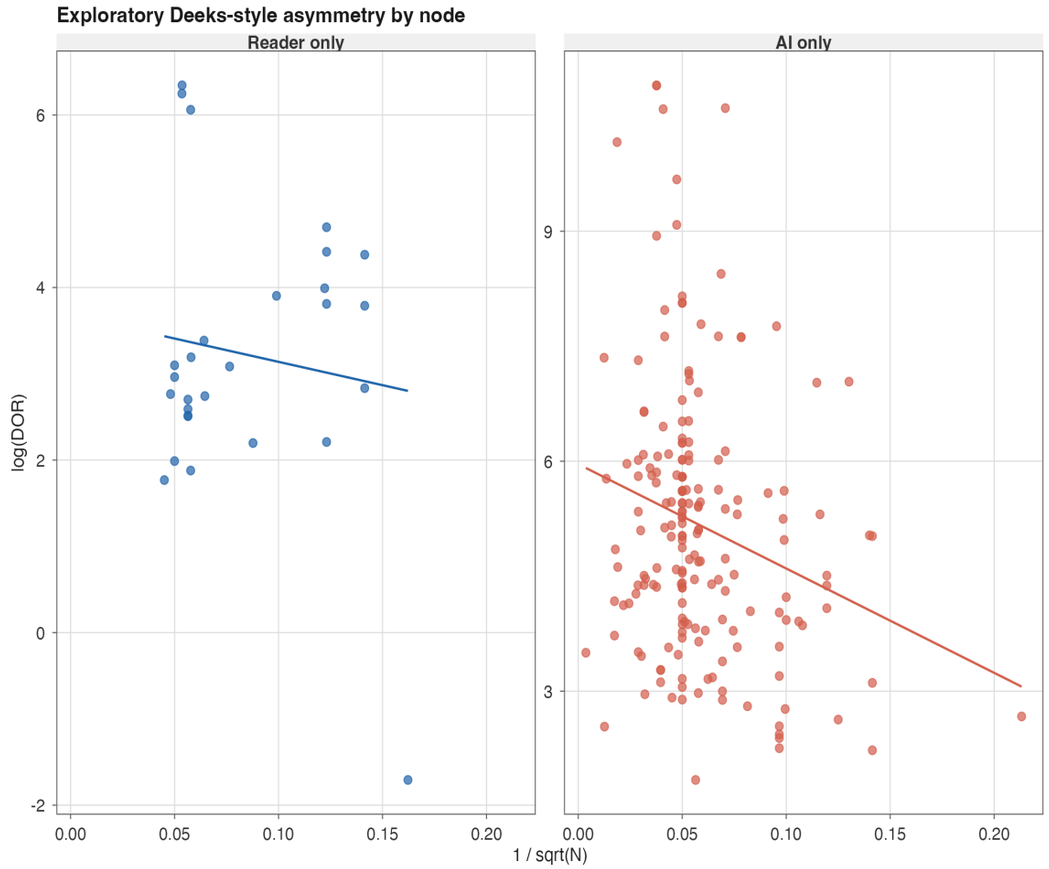
A non-zero slope suggests small studies report systematically different effects — usually a **publication bias** signal.

> **Note**
>
> Standard funnel plots assume effect-size symmetry. **Deeks** is the DTA-appropriate variant.

Deeks funnel — our study



eFigure 11. Pooled funnel.



eFigure 12. Per-node funnel.

Bonus: Building this with Claude / Codex

How a clinician with limited R fluency goes from a blank file to a publication-grade pipeline — using an AI coding assistant as a pair programmer.

Why use an AI coding assistant?

- **You know the clinical question and the statistics.** You don’t need to memorise every `dplyr` verb or JAGS syntax quirk.
- **You stay the author of the analysis.** The model writes code; *you* read it, run it, sanity-check it, and decide what to keep.
- **It accelerates the unglamorous parts:** harmonisation, plotting boilerplate, reshaping data, writing the same forest-plot code for the 14th time.
- **It’s a reading aid:** paste a JAGS block, ask “explain this line by line” — turns opaque code into a teachable artefact.

> ⚠ **Warning**

The two assistants — and what to subscribe to

	Claude (Anthropic)	Codex (OpenAI)
Best chat UI	claude.ai	chatgpt.com
Terminal CLI	**Claude Code** (recommended)	**Codex CLI**
Subscription tier needed	Claude Pro **or** Max	ChatGPT Plus **or** Pro
Monthly cost (USD)	~$20 (Pro) / ~$100 (Max)	~$20 (Plus) / ~$200 (Pro)
Strength	long-context analysis, agentic edits	fast iteration on snippets
File-system access	Yes (CLI in your terminal)	Yes (Codex CLI)

For a project like this (large workbook, multi-file pipeline, iterative figure work) — terminal CLI access matters more than the chat UI. Either Claude Code or Codex CLI does the job.

Setup in 5 minutes — Claude Code

```
# 1. Subscribe to Claude Pro at claude.ai (one-time)

npm install -g @anthropic-ai/claude-code   # 2. install CLI

cd ~/projects/glaucoma-dta-nma             # 3. enter your project

claude                                     # 4. launch — first run authenticates via browser
```

```
# Inside the session — just type. No commands to memorise.
> read 06_Secondary_AllValid and write me a transitivity table per node
> plot the HSROC with credible ellipses in Lancet colours
> my chains aren't converging — what should I check?
```

Note

The CLI sees your files. It can read the workbook, edit your `.Rmd`, run R scripts, and check the output — all in one loop.

Setup in 5 minutes — Codex CLI

```
# 1. Subscribe to ChatGPT Plus at chatgpt.com (one-time)

npm install -g @openai/codex                # 2. install CLI

cd ~/projects/glaucoma-dta-nma              # 3. enter your project

codex                                       # 4. launch — sign in via browser
```

Tip

Either tool works. I use Claude Code for long sessions (1M-context, better at multi-file edits) and Codex for quick one-off snippets. Subscribe to whichever UI you already prefer.

The four prompts that build 80% of the pipeline

1. **“Read `<workbook.xlsx>`, list the sheets, and tell me what each contains.”** → understands the data shape before writing any code.
2. **“Harmonise the `modality` column into Fundus / OCT / Fundus+OCT / Mixed using `case_when` + `str_detect`. Show me the unique input strings first.”** → forces it to inspect the raw data, not hallucinate categories.
3. **“Write a Bayesian bivariate DTA model in JAGS for arm-based pooling, two test types, with a study-level bivariate normal random effect and Se–Sp correlation. Use weakly informative priors.”** → generates the model spec verbatim. Validate against Doebler & Holling.
4. **“Make a publication-grade forest plot with credible AND prediction intervals, Lancet palette, in `ggplot2`.”** → handles the plotting boilerplate.

A real example — debugging convergence

Your prompt:

> *$\hat{R}$ for `SDstudysens` is 1.18 after 20k iterations, 4 chains. Trace plot shows one chain stuck. What should I try, in order of likelihood?*

What you get back:

1. Tighter init values for `SDstudysens` (e.g. `runif(0.1, 0.5)` not `runif(0, 2)`)
2. Reparameterise: half-Cauchy on SD instead of Uniform(0,2)
3. Increase burn-in to 20k
4. Check for a single influential study via LOO

Important

The judgment is still yours. The model offers candidates ranked by plausibility. You pick — and you're the one who reports the choice in Methods.

Workflow rules of thumb

What to *never* delegate

- The **PICO** and **inclusion criteria** — yours alone.
- The **risk-of-bias judgement** for each study — yours alone (or a second human reviewer).
- The **decision** on which sensitivity analyses are pre-specified vs post-hoc.
- The **interpretation** of Pr(AI > Reader) for clinicians.
- The **Methods write-up** — the AI can draft, you must verify every claim against the code that ran.

> ⚠ **Warning**
>
> **Reviewer-facing rule:** if you used an AI assistant, say so in the methods. Most journals now have a disclosure requirement; the wording is usually “AI assistance was used to draft and refactor R code; all statistical decisions and outputs were verified by the authors.”

Bringing it together

What we built — and what it took

Output	Code lines (approx.)	Reviewer-facing?
PRISMA flow	35	✓ Main
Cleaning + harmonisation	120	✗ Supplement
QUADAS-3 traffic light	50	✓ Main
Bayesian model + fit	80	✓ Methods
Forest + HSROC	90	✓ Main
Sensitivity / LOO / Deeks	150	✓ Supplement
Total	**~600 lines**	

Reusable across projects. This is the trilogy logic — the third NMA borrows ~70% from the first two.

Common pitfalls — and how the notebook handles them

- **Mixed units (eye vs patient vs image-level)** → explicit `cohort_id` and audit sheet
- **Missing 2×2 cells** → `filter(!if_any(c(tp,fn,tn,fp), is.na))`
- **Inconsistent vendor reporting** → three-level harmonised factor (Yes/No/Unclear)
- **MCMC non-convergence** → multiple init seeds, $\hat{R}$ checked programmatically
- **Cherry-picked subgroups** → pre-specified list, all reported
- **Forgotten heterogeneity** → prediction interval shown alongside credible interval

Reporting checklist

- **PRISMA-DTA 2018** — every flow box, every sheet
- **QUADAS-3** — traffic light + domain summary table
- **GRADE-DTA** — per-10,000 outcomes table
- **MCMC diagnostics** — $\hat{R}$ and trace plots in supplement
- **Code & data availability** — R Markdown + workbook on OSF / GitHub

> **Tip**
>
> **Habit:** create the output folder structure (`figures/`, `tables/`, `publication/main/`, `publication/supplement/`) at the *start* of the project. The notebook does this in `setup`.

What I want you to remember

1. **Harmonisation is half the work.** Get it right, automate it.
2. **Bivariate, hierarchical, Bayesian** — these aren't fashion words. Each one earns its place.
3. **Prediction intervals**, not just credible intervals.
4. **Pre-specified sensitivity analyses**, not post-hoc ones.
5. **Cache your model fits.** Future-you will thank you.
6. **Speak in 10,000-screened terms** when you present to clinicians.

Resources

- **JAGS manual** — Plummer, *JAGS Version 4.3.0 user manual*
- **R2jags** package documentation
- **Doebler & Holling** — bivariate DTA tutorials in R
- **Cochrane DTA Handbook** — Chapter 10 on meta-analysis methods
- **PRISMA-DTA 2018** — Salameh et al., BMJ
- **GRADE for DTA** — Schünemann et al., *J Clin Epidemiol*

Code & data: `https://github.com/[your-handle]/glaucoma-dta-nma`

Questions?

Thank you.

Slides built with Quarto + revealjs